

Advance Clustering Methods Final Project

Banira Acharya Dhakal, Segan Foster, Monique Handsom

2026-05-13

Introduction

Clustering is an unsupervised machine learning technique used to identify natural groups in data without using a predefined outcome variable. In healthcare data, clustering can help reveal hidden health profiles, risk patterns, and subgroups of individuals who share similar demographic, physiological, and metabolic characteristics.

This project applies clustering techniques to the National Health and Nutrition Examination Survey (NHANES) 2017–2018 dataset. The selected variables include age, sex, income-to-poverty ratio, body mass index (BMI), blood pressure, and glycohemoglobin (HbA1c). These variables capture important demographic and health-related characteristics. The main goal of the project is to identify meaningful health-related clusters, interpret their characteristics, and compare different clustering methods.

Brief Implementation Overview

The implementation follows five major steps. First, NHANES demographic, body measurement, blood pressure, glycohemoglobin, and questionnaire files are downloaded and merged using the participant identifier SEQN. Second, the selected variables are cleaned and prepared by checking missing values, applying median imputation to numeric variables, examining correlations, removing redundant variables, and scaling the final continuous clustering variables. Third, three clustering methods are applied: K-means, DBSCAN, and Gaussian Mixture Models (GMM). Fourth, model parameters are selected using validation metrics and diagnostic plots. Finally, the clustering results are evaluated and visualized using internal validation metrics, PCA plots, UMAP plots, silhouette plots, cluster profile tables, boxplots, a heatmap, and a parallel coordinate plot.

Packages and Data Collection

```
# Install packages if needed
packages <- c(
  "nhanesA", "tidyverse", "e1071", "knitr", "corrplot", "umap",
  "cluster", "clusterCrit", "clusterSim", "dbscan", "fpc", "mclust",
  "factoextra", "GGally", "viridis", "RColorBrewer"
)
```

```
new_packages <- packages[!(packages %in% installed.packages()[, "Package"])]
if(length(new_packages) > 0) install.packages(new_packages)
```

```
library(nhanesA)
library(tidyverse)
library(e1071)
library(knitr)
library(corrplot)
library(umap)
library(cluster)
library(clusterCrit)
library(clusterSim)
library(dbSCAN)
library(fpc)
library(mclust)
library(factoextra)
library(GGally)
library(gridExtra)
library(viridis)
library(RColorBrewer)
library(tidyr)
library(dplyr)
library(kableExtra)
```

```
# Load NHANES 2017-2018 data files
```

```
demo <- nhanes("DEMO_J")
bmx <- nhanes("BMX_J")
bpx <- nhanes("BPX_J")
ghb <- nhanes("GHB_J")
bpq <- nhanes("BPQ_J")
```

```
# Merge data files using participant ID
```

```
dataset <- demo %>%
  inner_join(bmx, by = "SEQN") %>%
  inner_join(bpx, by = "SEQN") %>%
  inner_join(ghb, by = "SEQN") %>%
  inner_join(bpq, by = "SEQN")
```

```
# Select variables for analysis
```

```
datafeat <- dataset %>%
  dplyr::select(
    age      = RIDAGEYR,
    sex      = RIAGENDR,
    income_pir = INDFMPIR,
    bmi      = BMXBMI,
    waist    = BMXWAIST,
    sys_bp   = BPXSY1,
    dia_bp   = BPXDI1,
```

```

    hba1c      = LBXGH
  )

dim(datafeat)

## [1] 5828    8

```

Exploratory Data Analysis and Preprocessing

Exploratory data analysis was conducted to understand the structure and quality of the selected NHANES variables before clustering. Summary statistics were calculated to examine the central tendency, spread, and skewness of the variables. Histograms and boxplots were used to examine distributions and potential outliers. Missing values were identified and treated using median imputation for numeric variables. Median imputation was selected because it is less sensitive to extreme values than mean imputation.

A correlation matrix was also created to examine relationships among variables. BMI and waist circumference were highly correlated, so waist circumference was removed from the final clustering variables to avoid redundancy. Sex was also excluded from the clustering model because it is categorical and not appropriate for direct use in distance-based clustering without additional encoding. However, sex was retained later for cluster profiling and interpretation. The final clustering variables were standardized so that variables measured on larger scales did not dominate the clustering results.

Summary Statistics Before Imputation

```

summary_stats <- datafeat %>%
  summarise(across(where(is.numeric),
    list(
      mean = ~mean(.x, na.rm = TRUE),
      median = ~median(.x, na.rm = TRUE),
      variance = ~var(.x, na.rm = TRUE),
      skewness = ~skewness(.x, na.rm = TRUE)
    ),
    .names = "{.col}_{.fn}"))

summary_stats %>%
  pivot_longer(
    cols = everything(),
    names_to = c("Variable", "Statistic"),
    names_pattern = "(.+)_ (mean|median|variance|skewness)"
  ) %>%
  pivot_wider(names_from = Statistic, values_from = value) %>%
  mutate(across(where(is.numeric), ~round(.x, 3))) %>%
  kable(caption = "Summary Statistics Before Imputation")

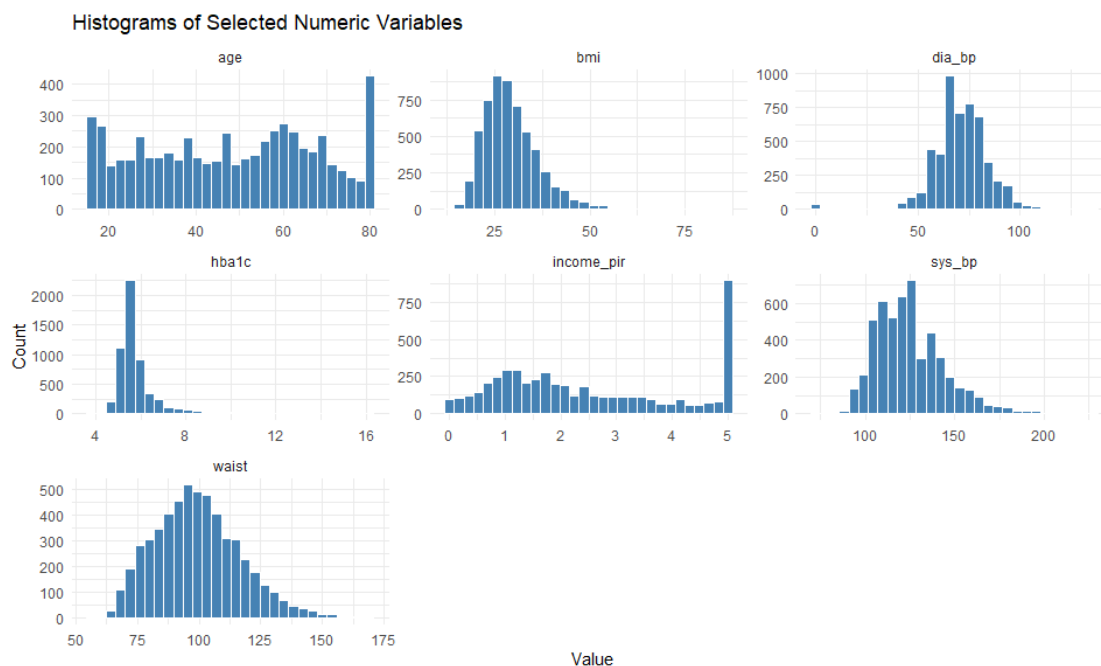
```

Summary Statistics Before Imputation

Variable	mean	median	variance	skewness
age	48.130	49.00	383.592	-0.040
income_pir	2.512	2.07	2.584	0.353
bmi	29.462	28.30	55.752	1.245
waist	99.336	98.10	306.631	0.469
sys_bp	125.134	122.00	387.482	1.034
dia_bp	71.198	72.00	185.042	-0.898
hba1c	5.811	5.50	1.148	3.239

Histograms

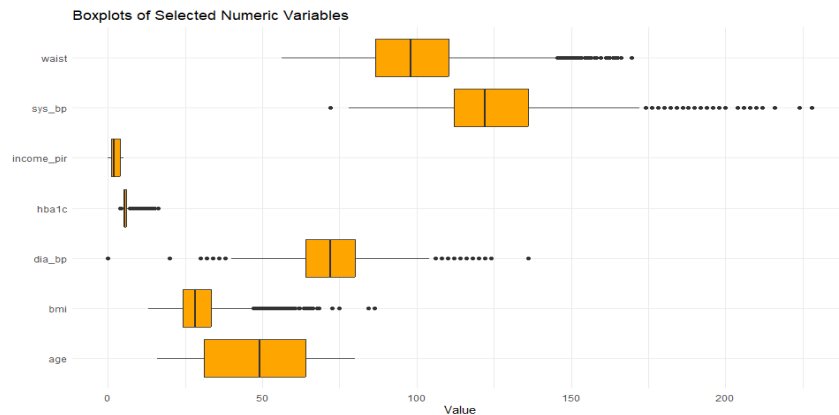
```
datafeat %>%  
  pivot_longer(cols = where(is.numeric)) %>%  
  ggplot(aes(x = value)) +  
  geom_histogram(bins = 30, fill = "steelblue", color = "white") +  
  facet_wrap(~name, scales = "free") +  
  theme_minimal() +  
  labs(title = "Histograms of Selected Numeric Variables", x = "Value", y =  
"Count")
```



Boxplots for Outlier Detection

```
datafeat %>%  
  pivot_longer(cols = where(is.numeric)) %>%  
  ggplot(aes(x = name, y = value)) +  
  geom_boxplot(fill = "orange") +
```

```
coord_flip() +
theme_minimal() +
labs(title = "Boxplots of Selected Numeric Variables", x = NULL, y =
"Value")
```



Missing Value Summary

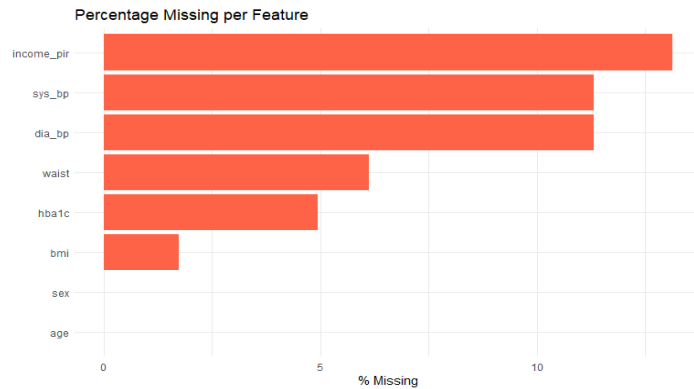
```
missing_summary <- datafeat %>%
  summarise(across(everything(), ~sum(is.na(.x)))) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "n_missing")
%>%
  mutate(pct_missing = round(n_missing / nrow(datafeat) * 100, 2))

kable(missing_summary, caption = "Missing Values by Variable")
```

Missing Values by Variable

variable	n_missing	pct_missing
age	0	0.00
sex	0	0.00
income_pir	765	13.13
bmi	101	1.73
waist	357	6.13
sys_bp	659	11.31
dia_bp	659	11.31
hba1c	288	4.94

```
ggplot(missing_summary, aes(x = reorder(variable, pct_missing), y =
pct_missing)) +
  geom_col(fill = "tomato") +
  coord_flip() +
  labs(title = "Percentage Missing per Feature", x = NULL, y = "% Missing") +
  theme_minimal()
```



Median Imputation

```
data_imputed <- datafeat %>%
  mutate(across(where(is.numeric), ~ifelse(is.na(.x), median(.x, na.rm =
TRUE), .x)))

summary_stats_imputed <- data_imputed %>%
  summarise(across(where(is.numeric),
    list(
      mean = ~mean(.x, na.rm = TRUE),
      median = ~median(.x, na.rm = TRUE),
      min = ~min(.x, na.rm = TRUE),
      max = ~max(.x, na.rm = TRUE)
    ),
    .names = "{.col}_{.fn}")) %>%
  pivot_longer(
    cols = everything(),
    names_to = c("Variable", "Statistic"),
    names_pattern = "(.+)_ (mean|median|min|max)",
    values_to = "value"
  ) %>%
  pivot_wider(names_from = Statistic, values_from = value) %>%
  mutate(across(where(is.numeric), ~round(.x, 2)))

kable(summary_stats_imputed, caption = "Summary Statistics After Median
Imputation")
```

Summary Statistics After Median Imputation

Variable	mean	median	min	max
age	48.13	49.00	16.0	80.0
income_pir	2.45	2.07	0.0	5.0
bmi	29.44	28.30	13.2	86.2
waist	99.26	98.10	56.4	169.5
sys_bp	124.78	122.00	72.0	228.0

Variable	mean	median	min	max
dia_bp	71.29	72.00	0.0	136.0
hba1c	5.80	5.50	3.8	16.2

Correlation Matrix and Feature Selection

```
cor_matrix <- data_imputed %>%
  dplyr::select(where(is.numeric)) %>%
  cor()

corrplot(cor_matrix, method = "color", tl.cex = 0.8, addCoef.col = "black",
number.cex = 0.6)
```



```
high_corr <- which(abs(cor_matrix) > 0.8 & abs(cor_matrix) < 1, arr.ind =
TRUE)
high_corr

##      row col
## waist  4   3
## bmi    3   4
```

The correlation analysis was used to identify redundancy among the selected variables. Because BMI and waist circumference measure closely related body-size characteristics and were highly correlated, waist circumference was removed from the final clustering model. This helps reduce duplicated information in the clustering solution.

Final Clustering Dataset and Scaling

```
# Remove sex because it is categorical.
# Remove waist because it is highly correlated with BMI.
data_cluster <- data_imputed %>%
  dplyr::select(-sex, -waist)

# Check variances before scaling
apply(data_cluster, 2, var)
```

```
##      age income_pir      bmi      sys_bp      dia_bp      hba1c
## 383.592104  2.267213  54.808526 344.645463 164.178919  1.095336

# Scale numeric data for clustering
data_scaled <- scale(data_cluster)

# Confirm no missing values after scaling
colSums(is.na(data_scaled))

##      age income_pir      bmi      sys_bp      dia_bp      hba1c
##      0           0           0           0           0           0
```

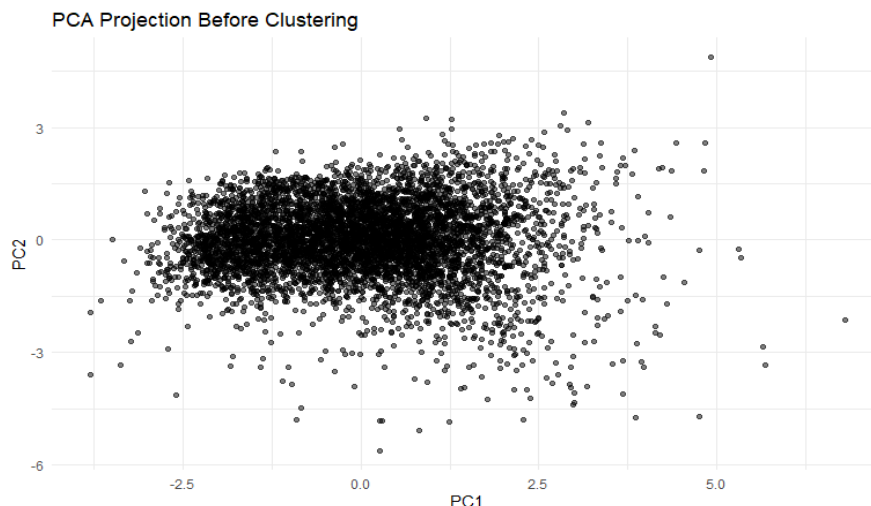
Scaling is necessary because the selected variables are measured on different scales. For example, age, blood pressure, BMI, and HbA1c have different units and ranges. Standardization gives each variable a mean of 0 and a standard deviation of 1, allowing each variable to contribute more equally to distance-based clustering.

Dimensionality Reduction and Visual Exploration

Dimensionality reduction was used to visually examine possible cluster structure before applying clustering algorithms. PCA was used to project the standardized data into two dimensions while preserving as much total variation as possible. UMAP was also used to create a nonlinear two-dimensional visualization that can better capture local relationships among observations.

```
# PCA visualization before clustering
pca_result <- prcomp(data_scaled)
pca_data <- as.data.frame(pca_result$x)

ggplot(pca_data, aes(x = PC1, y = PC2)) +
  geom_point(alpha = 0.5) +
  theme_minimal() +
  labs(title = "PCA Projection Before Clustering", x = "PC1", y = "PC2")
```




```
# UMAP visualization before clustering
set.seed(123)
umap_result <- umap(data_scaled)
umap_data <- as.data.frame(umap_result$layout)
colnames(umap_data) <- c("UMAP1", "UMAP2")

ggplot(umap_data, aes(x = UMAP1, y = UMAP2)) +
  geom_point(alpha = 0.5) +
  theme_minimal() +
  labs(title = "UMAP Projection Before Clustering", x = "UMAP1", y = "UMAP2")
```



Methodology: Clustering Methods, Parameters, Metrics, and Visualizations

Three clustering methods were implemented and compared: K-means, DBSCAN, and Gaussian Mixture Models. K-means was selected because it is simple, widely used, easy to interpret, and suitable for continuous numeric variables. DBSCAN was selected because it can identify dense clusters and label unusual observations as noise. GMM was selected because it allows probability-based cluster assignment and is useful when observations may share characteristics with multiple groups.

The main tuning parameter for K-means is the number of clusters, K. Several K values were evaluated using the Elbow Method, Silhouette Score, Davies-Bouldin Index, and Calinski-Harabasz Index. For DBSCAN, the main parameters are `eps` and `minPts`; these were tuned using a k-distance plot and a grid search. For GMM, the number of mixture components and covariance structure were selected using BIC and internal validation metrics.

The clustering models were evaluated using Silhouette Score, Davies-Bouldin Index, Calinski-Harabasz Index, cluster size distribution, and practical interpretability. Visualizations included PCA plots, UMAP plots, validation metric plots, silhouette plots, heatmaps, boxplots, and parallel coordinate plots.

DBSCAN

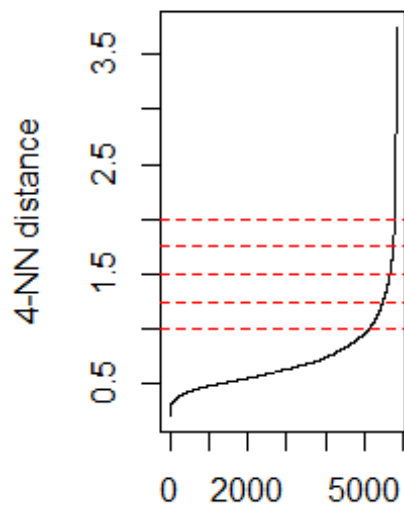
DBSCAN (Density Based Spatial Clustering of Applications with Noise) was chosen as one of the clustering methods because its method does not require a cluster number to be known in advance. It is also able to categorize outliers as noise points. This is very valuable when dealing with healthcare data as there are often cases with very extreme values associated with them and are often wanted for identification for study and recognition.

K Distance plot is used to guide finding the epsilon value. The plot sorts all observations by their distance to their k-th nearest neighbor. At the elbow, or curve bend, the natural density threshold and starting point for the epsilon value is indicated. To check sensitivity of the elbow to the choice of k, the plot is generated at two values of k.

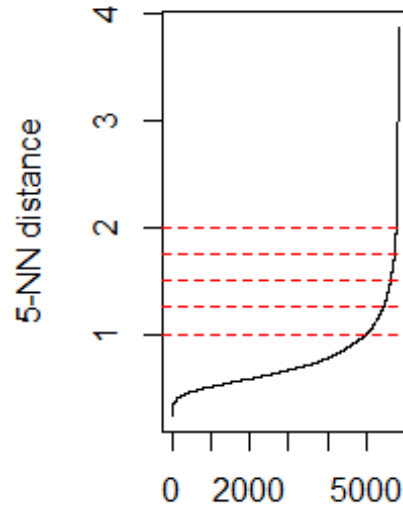
```
par(mfrow = c(1, 2))
```

```
kNNdistplot(data_scaled, k = 4)
abline(h = 1.0, col = "red", lty = 2)
abline(h = 1.25, col = "red", lty = 2)
abline(h = 1.5, col = "red", lty = 2)
abline(h = 1.75, col = "red", lty = 2)
abline(h = 2.0, col = "red", lty = 2)
title("K-Distance Plot (k = 4)")
```

```
kNNdistplot(data_scaled, k = 5)
abline(h = 1.0, col = "red", lty = 2)
abline(h = 1.25, col = "red", lty = 2)
abline(h = 1.5, col = "red", lty = 2)
abline(h = 1.75, col = "red", lty = 2)
abline(h = 2.0, col = "red", lty = 2)
title("K-Distance Plot (k = 5)")
```

K-Distance Plot (k = 4)

Points sorted by distance

K-Distance Plot (k = 5)

Points sorted by distance

```
par(mfrow = c(1, 1))
```

The K-Distance plots were used to guide the selection of epsilon. Both plots show an elbow at an approximate value of 1.5, which is consistent across the two k values tested. This consistency indicates the elbow is not an artifact of the choice of k, and suggests the natural density threshold lies at around 1.5.

To clarify what combination of epsilon and minPts values would best suit the data, a grid search is conducted. The epsilon values 1.2 to 1.7 and minPts values from 3 to 7 will be calculated.

```
eps.vals      <- seq(1.2, 1.7, by = 0.1)
minpts.vals   <- c(3, 4, 5, 6, 7)

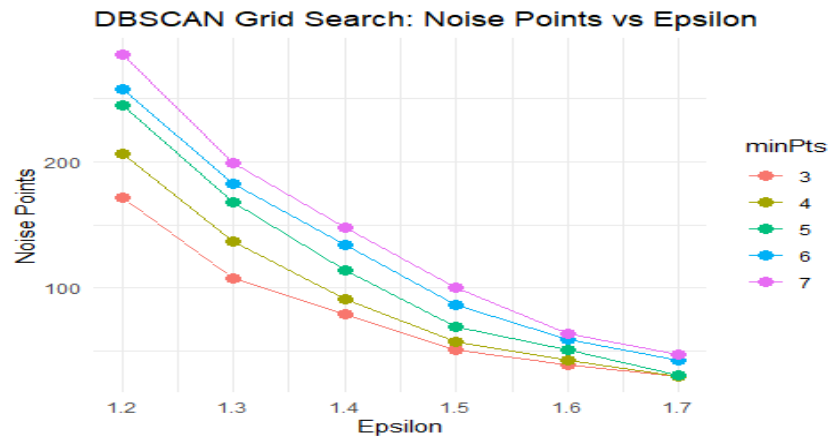
grid.results  <- expand.grid(eps = eps.vals, minPts = minpts.vals)
grid.results$n_clusters <- NA
grid.results$n_noise   <- NA

for (i in seq_len(nrow(grid.results))) {
  fit <- dbSCAN::dbSCAN(data_scaled,
                        eps      = grid.results$eps[i],
                        minPts   = grid.results$minPts[i])
  grid.results$n_clusters[i] <- max(fit$cluster)
  grid.results$n_noise[i]    <- sum(fit$cluster == 0)
}
```

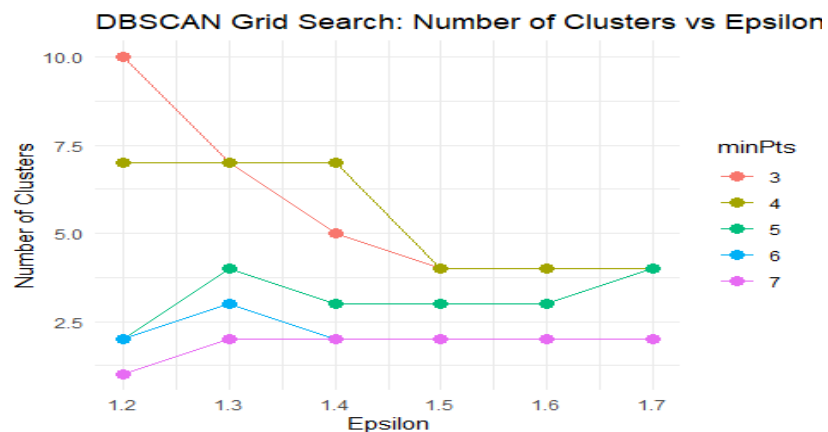
```
print(grid.results)
```

```
##      eps minPts n_clusters n_noise
## 1  1.2      3      10      172
## 2  1.3      3       7      108
## 3  1.4      3       5       79
## 4  1.5      3       4       51
## 5  1.6      3       4       39
## 6  1.7      3       4       30
## 7  1.2      4       7      206
## 8  1.3      4       7      137
## 9  1.4      4       7       91
## 10 1.5      4       4       57
## 11 1.6      4       4       43
## 12 1.7      4       4       30
## 13 1.2      5       2      245
## 14 1.3      5       4      168
## 15 1.4      5       3      114
## 16 1.5      5       3       69
## 17 1.6      5       3       51
## 18 1.7      5       4       31
## 19 1.2      6       2      258
## 20 1.3      6       3      183
## 21 1.4      6       2      134
## 22 1.5      6       2       87
## 23 1.6      6       2       59
## 24 1.7      6       2       43
## 25 1.2      7       1      285
## 26 1.3      7       2      199
## 27 1.4      7       2      148
## 28 1.5      7       2      100
## 29 1.6      7       2       64
## 30 1.7      7       2       47
```

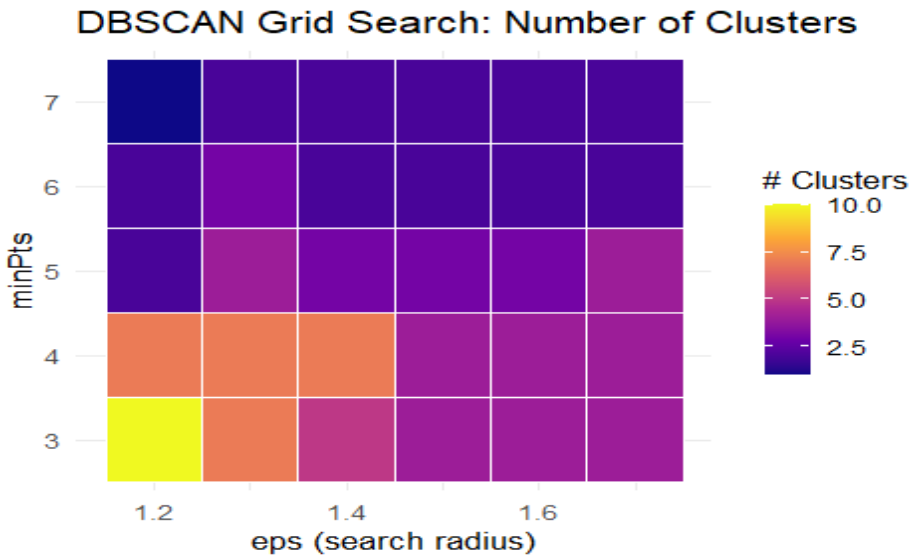
```
ggplot(grid.results, aes(x = eps, y = n_noise,
                        colour = as.factor(minPts),
                        group = as.factor(minPts))) +
  geom_line() +
  geom_point(size = 3) +
  theme_minimal() +
  labs(title = "DBSCAN Grid Search: Noise Points vs Epsilon",
       x = "Epsilon",
       y = "Noise Points",
       colour = "minPts")
```



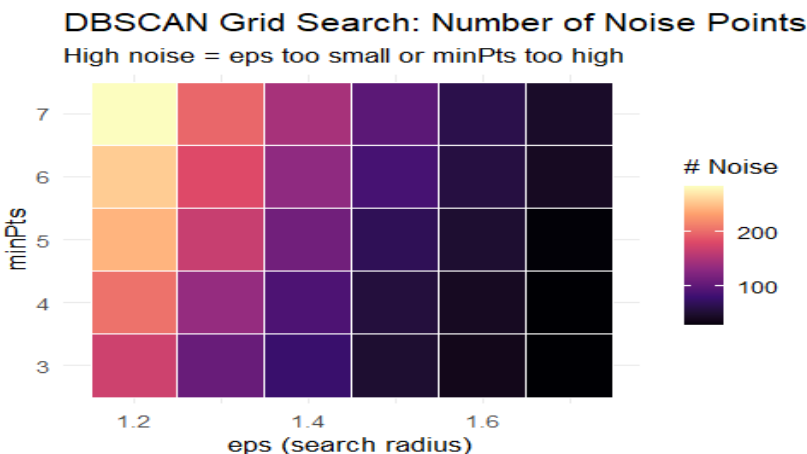
```
ggplot(grid.results, aes(x = eps, y = n_clusters,
                        colour = as.factor(minPts),
                        group = as.factor(minPts))) +
  geom_line() +
  geom_point(size = 3) +
  theme_minimal() +
  labs(title = "DBSCAN Grid Search: Number of Clusters vs Epsilon",
       x = "Epsilon",
       y = "Number of Clusters",
       colour = "minPts")
```



```
# Heatmap: number of clusters at each parameter combination
ggplot(grid.results, aes(x = eps, y = factor(minPts), fill = n_clusters)) +
  geom_tile(color = "white") +
  scale_fill_viridis_c(option = "plasma", name = "# Clusters") +
  labs(title = "DBSCAN Grid Search: Number of Clusters",
       x = "eps (search radius)",
       y = "minPts") +
  theme_minimal(base_size = 13)
```



```
# Heatmap: number of noise points at each parameter combination
ggplot(grid.results, aes(x = eps, y = factor(minPts), fill = n_noise)) +
  geom_tile(color = "white") +
  scale_fill_viridis_c(option = "magma", name = "# Noise") +
  labs(title = "DBSCAN Grid Search: Number of Noise Points",
       subtitle = "High noise = eps too small or minPts too high",
       x = "eps (search radius)",
       y = "minPts") +
  theme_minimal(base_size = 13)
```



A grid search was conducted across a range of epsilon values from 1.2 to 1.7 and minPts values of 3 to 7. Although it subverts the standard rule of minPts being the number of dimensions plus one, a minPts of 5 was chosen. This is because the expected minPts = 7 consistently produced only one or two clusters when tested. At eps 1.5 and minPts 5, DBSCAN produced a stable three cluster solution with 69 noise points (1.18%). An eps of 1.6 was rejected because of the apparent instability of its neighbor 1.7 that produced 4 clusters. In contrast, 1.5 occupies a stable position of three clusters on both sides.

Compare candidate DBSCAN fits
Following an exploratory tight, medium, and strict comparison, three
candidate parameter combinations are fit to confirm sensitivity of the chosen
solution before committing to a final model.

```
set.seed(123)
```

```
db.tight <- dbSCAN::dbSCAN(data_scaled, eps = 1.3, minPts = 5)
db.mid   <- dbSCAN::dbSCAN(data_scaled, eps = 1.5, minPts = 5)
db.strict <- dbSCAN::dbSCAN(data_scaled, eps = 1.5, minPts = 7)
```

```
cat("\n--- DBSCAN Tight (eps = 1.3, minPts = 5) ---\n")
```

```
##
```

```
## --- DBSCAN Tight (eps = 1.3, minPts = 5) ---
```

```
print(table(db.tight$cluster))
```

```
##
```

```
##    0    1    2    3    4
## 168 5640   10    5    5
```

```
cat("Noise points:", sum(db.tight$cluster == 0), "\n\n")
```

```
## Noise points: 168
```

```
cat("--- DBSCAN Medium (eps = 1.5, minPts = 5) ---\n")
```

```
## --- DBSCAN Medium (eps = 1.5, minPts = 5) ---
```

```
print(table(db.mid$cluster))
```

```
##
```

```
##    0    1    2    3
##   69 5728   26    5
```

```
cat("Noise points:", sum(db.mid$cluster == 0), "\n\n")
```

```
## Noise points: 69
```

```
cat("--- DBSCAN Strict (eps = 1.5, minPts = 7) ---\n")
```

```
## --- DBSCAN Strict (eps = 1.5, minPts = 7) ---
```

```
print(table(db.strict$cluster))
```

```
##
```

```
##    0    1    2
## 100 5707   21
```

```
cat("Noise points:", sum(db.strict$cluster == 0), "\n")
```

```
## Noise points: 100
```

```

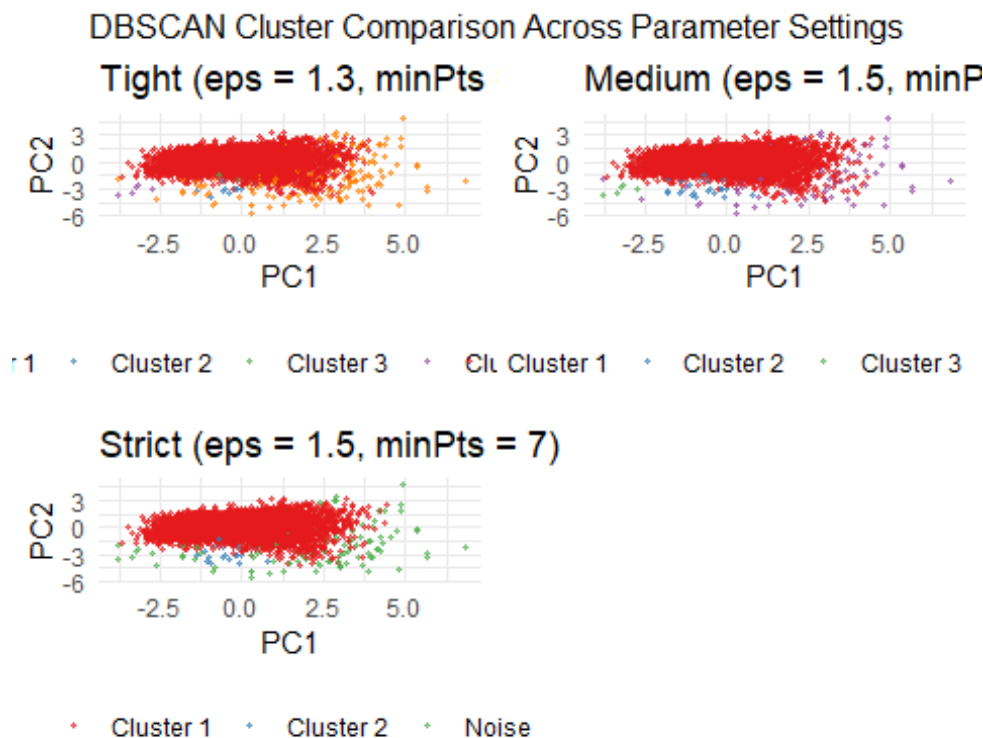
# Side-by-side visualization of candidate fits in PCA space
plot_dbscan <- function(clusters, title_str, pca_df = pca_data) {
  df      <- pca_df
  df$clus <- factor(ifelse(clusters == 0,
                           "Noise",
                           paste("Cluster", clusters)))

  ggplot(df, aes(PC1, PC2, color = clus)) +
    geom_point(size = 0.8, alpha = 0.6) +
    scale_color_brewer(palette = "Set1") +
    labs(title = title_str, color = NULL) +
    theme_minimal(base_size = 11) +
    theme(legend.position = "bottom")
}

p1 <- plot_dbscan(db.tight$cluster, "Tight (eps = 1.3, minPts = 5)")
p2 <- plot_dbscan(db.mid$cluster, "Medium (eps = 1.5, minPts = 5)")
p3 <- plot_dbscan(db.strict$cluster, "Strict (eps = 1.5, minPts = 7)")

grid.arrange(p1, p2, p3, ncol = 2,
              top = "DBSCAN Cluster Comparison Across Parameter Settings")

```



The tight setting (eps = 1.3) labels noticeably more observations as noise, the strict setting (minPts = 7) collapses the smaller dense regions into the dominant cluster, and the medium setting preserves the three-cluster structure observed in the grid search. The medium configuration is therefore retained as the final model.


```

# Fit DBSCAN Model
# eps = 1.5, minPts = 5, clusters = 3

set.seed(123)
dbscan.model <- dbscan::dbscan(data_scaled, eps = 1.5, minPts = 5)
# Define non-noise observations for DBSCAN evaluation
no.noise <- dbscan.model$cluster != 0

print(dbscan.model)

## DBSCAN clustering for 5828 objects.
## Parameters: eps = 1.5, minPts = 5
## Using euclidean distances and borderpoints = TRUE
## The clustering contains 3 cluster(s) and 69 noise points.
##
##      0      1      2      3
## 69 5728    26      5
##
## Available fields: cluster, eps, minPts, metric, borderPoints

cat("Cluster sizes:\n")

## Cluster sizes:

print(table(dbscan.model$cluster))

##
##      0      1      2      3
## 69 5728    26      5

cat("Noise points:", sum(dbscan.model$cluster == 0), "\n")

## Noise points: 69

cat("Noise percentage:",
    round(sum(dbscan.model$cluster == 0) / nrow(data_scaled) * 100, 2),
    "%\n")

## Noise percentage: 1.18 %

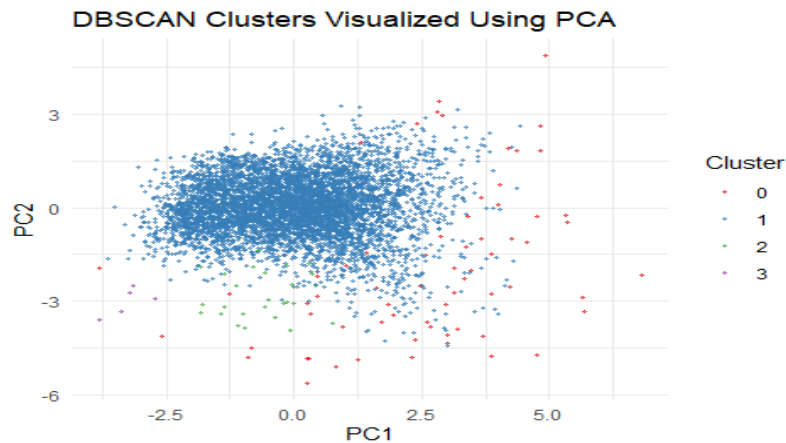
# Add cluster labels to data
data_cluster$dbscan_cluster <- as.factor(dbscan.model$cluster)

DBSCAN identified three clusters and one noise group. Cluster 1 contains 5,728
observations, over 95% of the data set. Clusters 2 and 3 are significantly smaller at 26 and
5 observations. This leaves 69 observations as noise in Cluster 0. This is not erroneous
data but extreme clinical cases that fall outside the minimum density threshold of the
other clusters.

# Visualization of DBSCAN in Reduced Dimensional Space
# DBSCAN PCA Visual
pca_data$dbscan_cluster <- as.factor(dbscan.model$cluster)

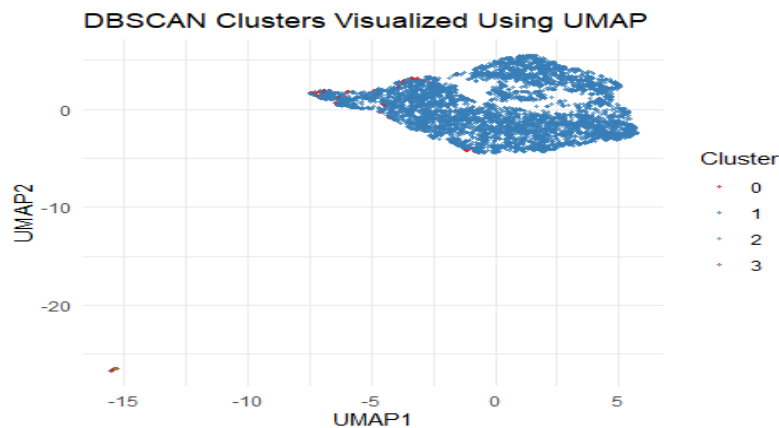
```

```
ggplot(pca_data, aes(x = PC1, y = PC2, color = dbscan_cluster)) +
  geom_point(alpha = 0.6, size = 0.8) +
  theme_minimal() +
  labs(title = "DBSCAN Clusters Visualized Using PCA",
       x = "PC1", y = "PC2",
       color = "Cluster") +
  scale_color_brewer(palette = "Set1")
```



```
# DBSCAN UMAP Visual
umap_data$dbscan_cluster <- as.factor(dbscan.model$cluster)

ggplot(umap_data, aes(x = UMAP1, y = UMAP2, color = dbscan_cluster)) +
  geom_point(alpha = 0.6, size = 0.8) +
  theme_minimal() +
  labs(title = "DBSCAN Clusters Visualized Using UMAP",
       x = "UMAP1", y = "UMAP2",
       color = "Cluster") +
  scale_color_brewer(palette = "Set1")
```



Both visuals confirm the dominant cluster output. Cluster 1 contains the central mass of the data while the smaller clusters and noise observations surround it on the periphery.

```

# Evaluation of DBSCAN
# DBSCAN Silhouette Analysis
# Compute silhouette, excluding noise (Cluster 0)
sil.dbscan <- silhouette(
  dbscan.model$cluster[no.noise],
  dist(data_scaled[no.noise, ])
)

# Convert to data frame
sil.df <- as.data.frame(unclass(sil.dbscan))
names(sil.df) <- c("cluster", "neighbor", "sil_width")

# Overall mean silhouette score
cat("DBSCAN Mean Silhouette Score:",
    round(mean(sil.df$sil_width), 4), "\n\n")

## DBSCAN Mean Silhouette Score: 0.4811

# Per-cluster silhouette distribution
sil.per.cluster <- sil.df %>%
  group_by(cluster) %>%
  summarise(
    n = n(),
    mean_sil = round(mean(sil_width), 4),
    median_sil = round(median(sil_width), 4),
    min_sil = round(min(sil_width), 4),
    max_sil = round(max(sil_width), 4),
    pct_negative = round(mean(sil_width < 0) * 100, 2),
    .groups = "drop"
  )

sil.per.cluster %>%
  mutate(cluster = paste("Cluster", cluster)) %>%
  rename(
    Cluster = cluster,
    N = n,
    Mean = mean_sil,
    Median = median_sil,
    Min = min_sil,
    Max = max_sil,
    `% Negative` = pct_negative
  ) %>%
  kable(
    caption = "Per-Cluster Silhouette Distribution",
    align = c("l", "r", "r", "r", "r", "r", "r")
  )

```

Per-Cluster Silhouette Distribution

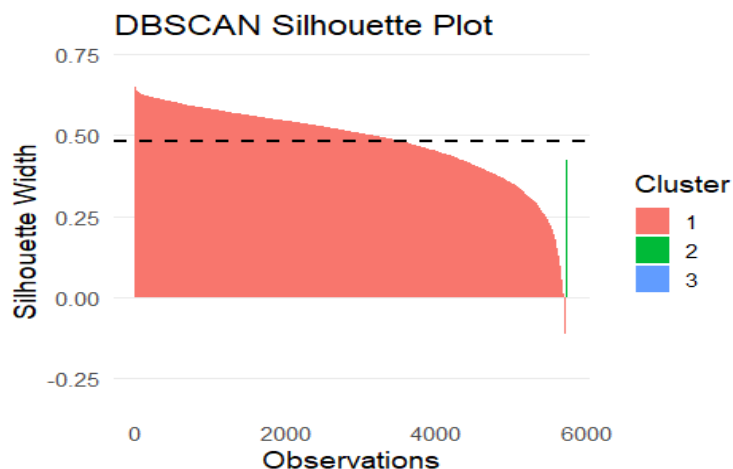
Cluster	N	Mean	Median	Min	Max	% Negative
---------	---	------	--------	-----	-----	------------

Cluster	N	Mean	Median	Min	Max	% Negative
Cluster 1	5728	0.4815	0.5109	-0.3074	0.6514	0.51
Cluster 2	26	0.3624	0.4103	0.0419	0.5259	0.00
Cluster 3	5	0.6315	0.6258	0.5743	0.7097	0.00

```
# Make plotting order
sil.df <- sil.df %>%
  arrange(cluster, desc(sil_width)) %>%
  mutate(id = row_number())

# Reference line for mean
avg.sil <- mean(sil.df$sil_width)

# DBSCAN Silhouette Plot
ggplot(sil.df, aes(x = id, y = sil_width, fill = factor(cluster))) +
  geom_col(width = 1) +
  geom_hline(yintercept = avg.sil, linetype = "dashed", linewidth = 1) +
  labs(
    title = "DBSCAN Silhouette Plot",
    x = "Observations",
    y = "Silhouette Width",
    fill = "Cluster"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    panel.grid.major.x = element_blank(),
    panel.grid.minor = element_blank(),
    legend.position = "right"
  )
```



A mean score of 0.48111 reflects moderate separation. The observations are ostensibly matched to their own cluster but boundaries are not readily defined, which is expected given that the vast majority of observations fall into a single cluster. Examined per cluster the distribution is uneven.

```

# DBSCAN Davies-Bouldin Index
# The Davies-Bouldin index is computed for further evaluation. Lower values
# indicate better separation.

db.index <- index.DB(
  data_scaled[no.noise, ],
  dbscan.model$cluster[no.noise]
)
cat("DBSCAN Davies-Bouldin Index:",
    round(db.index$DB, 4), "\n")

## DBSCAN Davies-Bouldin Index: 0.7432

# Cluster.stats output
db.stats <- cluster.stats(
  dist(data_scaled[no.noise, ]),
  dbscan.model$cluster[no.noise]
)

```

The Davies-Bouldin Index of 0.7432, where lower values denote more compact and better-separated clusters, falls into a moderate range.

```

# DBSCAN Calinski-Harabasz Index
cat("DBSCAN Calinski-Harabasz Index:",
    round(db.stats$ch, 4), "\n")

## DBSCAN Calinski-Harabasz Index: 96.6371

dbscan_metrics <- data.frame(
  Method = "DBSCAN",
  Silhouette = round(mean(sil.df$sil_width), 4),
  Davies_Bouldin = round(db.index$DB, 4),
  Calinski_Harabasz = round(db.stats$ch, 4)
)

kable(dbscan_metrics, caption = "Final DBSCAN Evaluation Metrics")

```

Final DBSCAN Evaluation Metrics

Method	Silhouette	Davies_Bouldin	Calinski_Harabasz
DBSCAN	0.4811	0.7432	96.6371

A value of 96.64 is modest. It reflects that DBSCAN found one overwhelmingly dominant cluster and two very small ones. This limits the degree of separation between clusters this metric can detect.

DBSCAN was applied across a comprehensive grid search of 30 parameter combinations and confirmed against three candidate fits (tight, medium, strict). In all cases the algorithm either produced one dominant cluster containing over 95% of observations alongside negligible splinter groups, or collapsed to two clusters with similar imbalance. No

parameter combination produced a meaningful multi-cluster solution with adequately sized groups. This indicates that the NHANES health profile data lacks the distinct structure DBSCAN requires. This finding itself is actually valuable. It suggests health risk profiles in this population exist on a continuous spectrum rather than forming isolated density regions, which means the use of GMM and K-Means are better suited to this data structure.

Gaussian Mixture Model Clustering

Gaussian Mixture Models were selected because they are useful when clusters may overlap. GMM assumes that the data are generated from a mixture of Gaussian distributions, where each distribution represents a cluster. Unlike K-means, which assigns each observation directly to one group, GMM estimates the probability that each observation belongs to each cluster.

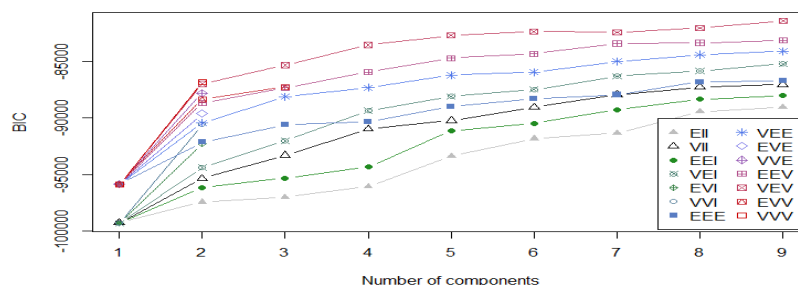
GMM Parameter Tuning

```
set.seed(123)

# Automatic GMM model selection using BIC
gmm_model <- Mclust(data_scaled)
summary(gmm_model)

## -----
## Gaussian finite mixture model fitted by EM algorithm
## -----
##
## Mclust VEV (ellipsoidal, equal shape) model with 9 components:
##
## log-likelihood    n df          BIC          ICL
##      -39771.76 5828 211  -81372.99 -84650.09
##
## Clustering table:
##      1      2      3      4      5      6      7      8      9
## 491  198  691  506  940 1166  544  775  517

plot(gmm_model, what = "BIC")
```



Davies-Bouldin and Calinski-Harabasz for GMM models with different numbers of clusters

```
g_range <- 2:9
```

```
gmm_validation <- data.frame(
  G = g_range,
  Model = NA,
  BIC = NA,
  AIC = NA,
  Silhouette = NA,
  Davies_Bouldin = NA,
  Calinski_Harabasz = NA
)
```

```
for (i in seq_along(g_range)) {
  g <- g_range[i]
  fit <- Mclust(data_scaled, G = g)
  clusters_gmm <- as.integer(fit$classification)

  sil <- silhouette(clusters_gmm, dist(data_scaled))
  gmm_validation$Silhouette[i] <- mean(sil[, 3])

  res <- intCriteria(as.matrix(data_scaled), clusters_gmm,
    c("Davies_Bouldin", "Calinski_Harabasz"))

  gmm_validation$Model[i] <- fit$modelName
  gmm_validation$BIC[i] <- fit$bic
  gmm_validation$AIC[i] <- -2 * fit$loglik + 2 * fit$df
  gmm_validation$Davies_Bouldin[i] <- res$davies_bouldin
  gmm_validation$Calinski_Harabasz[i] <- res$calinski_harabasz
}
```

```
gmm_validation <- gmm_validation %>%
  mutate(across(where(is.numeric), ~round(.x, 4)))
```

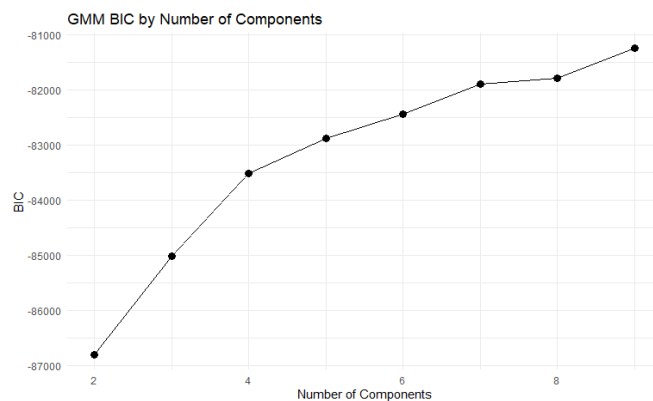
```
kable(gmm_validation, caption = "GMM Validation Metrics by Number of Components")
```

GMM Validation Metrics by Number of Components

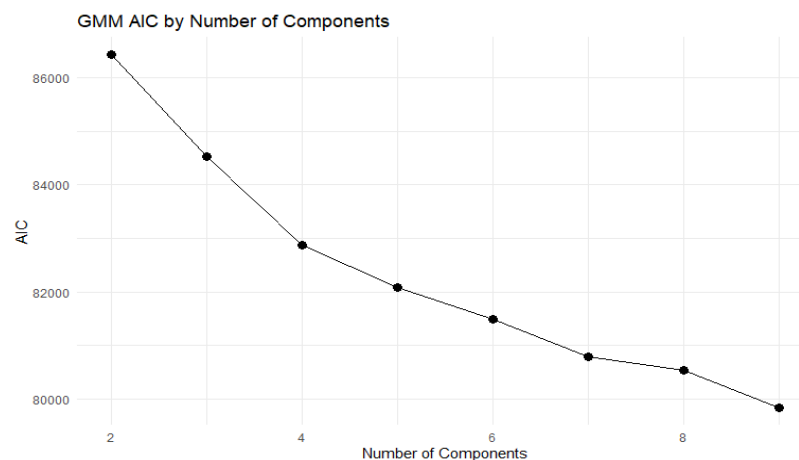
	Mod					
G	el	BIC	AIC	Silhouette	Davies_Bouldin	Calinski_Harabasz
2	VVV	-86790.89	86424.01	0.2461	2.1686	953.9958
3	VEV	-85008.67	84521.72	0.1667	1.8939	908.9528
4	VEV	-83512.20	82871.84	0.1813	1.8760	998.6605
5	VEV	-82872.22	82078.44	0.1234	2.0574	825.7462

	Mod					
G	el	BIC	AIC	Silhouette	Davies_Bouldin	Calinski_Harabasz
6	VEV	-82433.01	81485.81	0.0997	1.9826	721.0337
7	VEV	-81891.03	80790.41	0.0856	2.5829	631.8080
8	VEV	-81791.56	80537.52	0.0900	2.2003	607.6264
9	VEV	-81248.89	79841.42	0.0476	2.3250	521.5428

```
ggplot(gmm_validation, aes(x = G, y = BIC)) +
  geom_line() + geom_point(size = 3) +
  theme_minimal() +
  labs(title = "GMM BIC by Number of Components", x = "Number of Components",
y = "BIC")
```



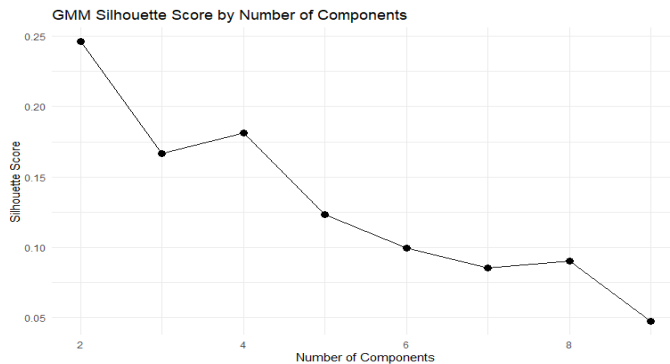
```
ggplot(gmm_validation, aes(x = G, y = AIC)) +
  geom_line() + geom_point(size = 3) +
  theme_minimal() +
  labs(title = "GMM AIC by Number of Components", x = "Number of Components",
y = "AIC")
```



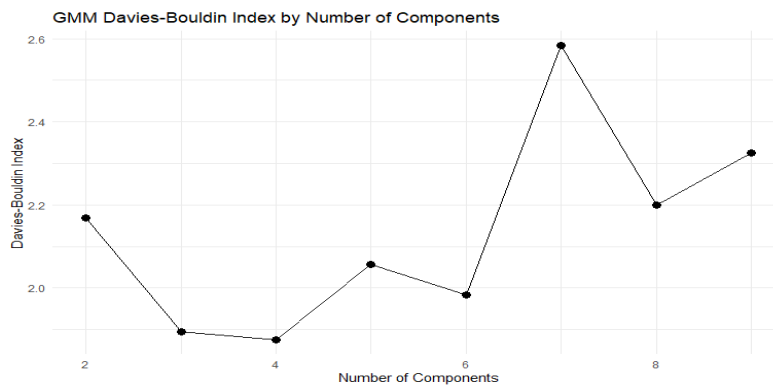
```
ggplot(gmm_validation, aes(x = G, y = Silhouette)) +
  geom_line() + geom_point(size = 3) +
  theme_minimal() +
```



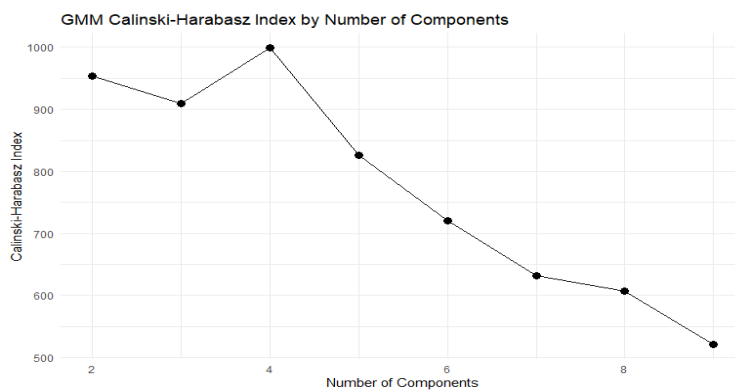
```
labs(title = "GMM Silhouette Score by Number of Components", x = "Number of Components", y = "Silhouette Score")
```



```
ggplot(gmm_validation, aes(x = G, y = Davies_Bouldin)) +  
  geom_line() + geom_point(size = 3) +  
  theme_minimal() +  
  labs(title = "GMM Davies-Bouldin Index by Number of Components", x =  
"Number of Components", y = "Davies-Bouldin Index")
```



```
ggplot(gmm_validation, aes(x = G, y = Calinski_Harabasz)) +  
  geom_line() + geom_point(size = 3) +  
  theme_minimal() +  
  labs(title = "GMM Calinski-Harabasz Index by Number of Components", x =  
"Number of Components", y = "Calinski-Harabasz Index")
```



Based on BIC, internal validation metrics, and interpretability, a four-cluster GMM solution with VEV covariance structure was selected. The VEV structure allows clusters to have variable volume, equal shape, and variable orientation, which gives the model more flexibility than a simple equal-variance model.

Final GMM Model and Evaluation

```
set.seed(123)
gmm_final <- Mclust(data_scaled, G = 4, modelNames = "VEV")

clusters_gmm_final <- as.integer(gmm_final$classification)

gmm_indices <- intCriteria(
  traj = as.matrix(data_scaled),
  part = clusters_gmm_final,
  crit = c("Davies_Bouldin", "Calinski_Harabasz")
)

gmm_sil <- silhouette(clusters_gmm_final, dist(data_scaled))
gmm_silhouette <- mean(gmm_sil[, 3])

gmm_metrics <- data.frame(
  Method = "GMM",
  Silhouette = round(gmm_silhouette, 4),
  Davies_Bouldin = round(gmm_indices$davies_bouldin, 4),
  Calinski_Harabasz = round(gmm_indices$calinski_harabasz, 4)
)

kable(gmm_metrics, caption = "Final GMM Evaluation Metrics")
```

Final GMM Evaluation Metrics

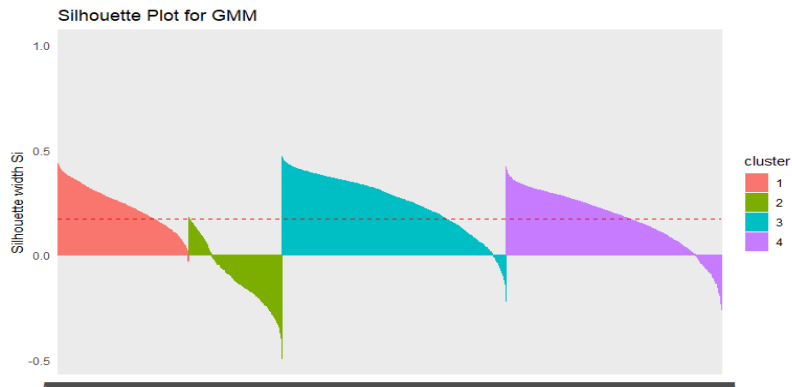
Method	Silhouette	Davies_Bouldin	Calinski_Harabasz
GMM	0.1746	1.9078	991.787

```
summary(gmm_sil)

## Silhouette of 5828 units in 4 clusters from silhouette.default(x =
## clusters_gmm_final, dist = dist(data_scaled)) :
## Cluster sizes and average silhouette widths:
##      1145      823      1970      1890
## 0.2395428 -0.1011910 0.2515514 0.1752064
## Individual silhouette widths:
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -0.49521 0.07651 0.21002 0.17462 0.31027 0.47092

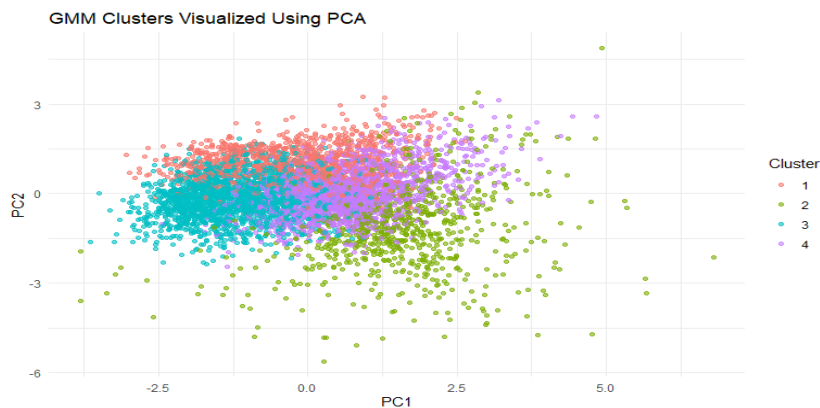
fviz_silhouette(gmm_sil) +
  labs(title = "Silhouette Plot for GMM") +
  theme_minimal()
```

```
##   cluster size ave.sil.width
## 1      1 1145      0.24
## 2      2  823     -0.10
## 3      3 1970      0.25
## 4      4 1890      0.18
```



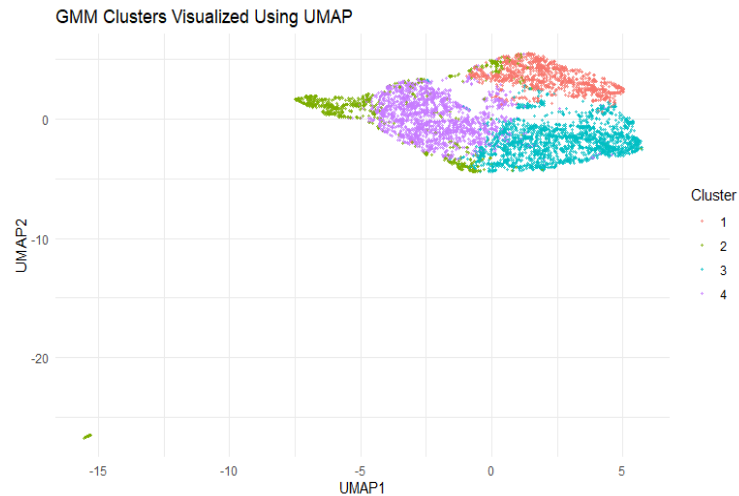
```
data_cluster$gmm_cluster <- as.factor(gmm_final$classification)
pca_data$gmm_cluster <- data_cluster$gmm_cluster
```

```
ggplot(pca_data, aes(PC1, PC2, color = gmm_cluster)) +
  geom_point(alpha = 0.6) +
  theme_minimal() +
  labs(title = "GMM Clusters Visualized Using PCA", x = "PC1", y = "PC2",
color = "Cluster")
```



```
umap_data$gmm_cluster <- data_cluster$gmm_cluster
```

```
ggplot(umap_data, aes(x = UMAP1, y = UMAP2, color = gmm_cluster)) +
  geom_point(alpha = 0.6, size = 0.8) +
  theme_minimal() +
  labs(title = "GMM Clusters Visualized Using UMAP", x = "UMAP1", y =
"UMAP2", color = "Cluster")
```



K-means Clustering

K-means clustering was applied to the standardized dataset. Conceptually, K-means assigns each observation to the nearest cluster center, called a centroid, and then updates the centroids until the assignments become stable. Because K-means requires the number of clusters to be selected in advance, several K values were tested before selecting the final model.

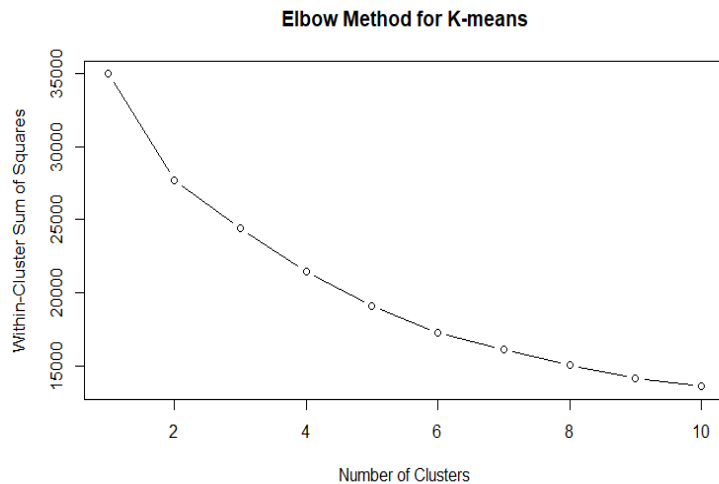
K-means Parameter Tuning

```
set.seed(123)
```

```
# Elbow Method
```

```
wss <- sapply(1:10, function(k) {  
  kmeans(data_scaled, centers = k, nstart = 25)$tot.withinss  
})
```

```
plot(1:10, wss, type = "b",  
     xlab = "Number of Clusters",  
     ylab = "Within-Cluster Sum of Squares",  
     main = "Elbow Method for K-means")
```



```
# Validation metrics for K = 2 to 10
validation_results <- data.frame(
  K = integer(),
  Silhouette_Score = numeric(),
  Davies_Bouldin_Index = numeric(),
  Calinski_Harabasz_Index = numeric()
)

for (k in 2:10) {
  set.seed(123)
  km <- kmeans(data_scaled, centers = k, nstart = 25)
  clusters <- km$cluster

  sil <- silhouette(clusters, dist(data_scaled))
  silhouette_score <- mean(sil[, 3])

  db <- intCriteria(as.matrix(data_scaled), clusters, "Davies_Bouldin")
  ch <- intCriteria(as.matrix(data_scaled), clusters, "Calinski_Harabasz")

  validation_results <- rbind(
    validation_results,
    data.frame(
      K = k,
      Silhouette_Score = silhouette_score,
      Davies_Bouldin_Index = db$davies_bouldin,
      Calinski_Harabasz_Index = ch$calinski_harabasz
    )
  )
}

validation_results <- validation_results %>%
  mutate(across(where(is.numeric), ~round(.x, 4)))
```

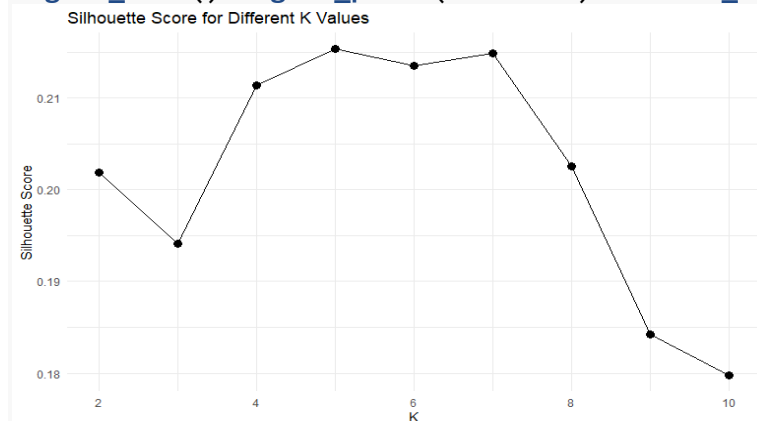
```
kable(validation_results, caption = "Validation Metrics for K-means")
```

Validation Metrics for K-means

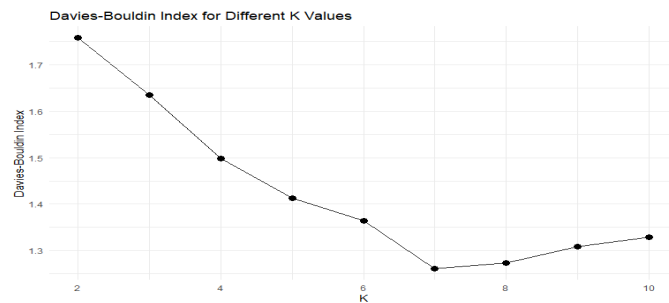
K	Silhouette_Score	Davies_Bouldin_Index	Calinski_Harabasz_Index
2	0.2019	1.7575	1534.722
3	0.1941	1.6355	1259.598
4	0.2114	1.4975	1223.244
5	0.2153	1.4129	1205.100
6	0.2135	1.3643	1195.268
7	0.2149	1.2617	1131.680
8	0.2025	1.2730	1101.250
9	0.1842	1.3089	1067.824
10	0.1798	1.3285	1015.720

```
ggplot(validation_results, aes(x = K, y = Silhouette_Score)) +
  geom_line() + geom_point(size = 3) + theme_minimal() +
  labs(title = "Silhouette Score for Different K Values", x = "K", y =
"Silhouette Score")
```

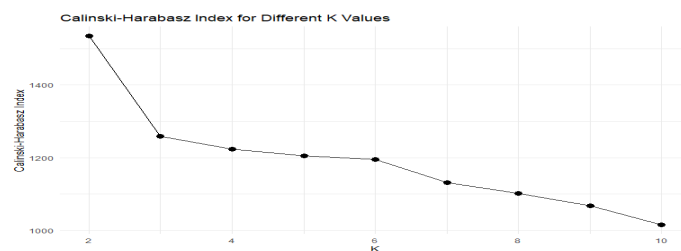
```
ggplot(validation_results, aes(x = K, y = Davies_Bouldin_Index)) +
  geom_line() + geom_point(size = 3) + theme_minimal() +
```



```
  labs(title = "Davies-Bouldin Index for Different K Values", x = "K", y =
"Davies-Bouldin Index")
```



```
ggplot(validation_results, aes(x = K, y = Calinski_Harabasz_Index)) +
  geom_line() + geom_point(size = 3) + theme_minimal() +
  labs(title = "Calinski-Harabasz Index for Different K Values", x = "K", y =
"Calinski-Harabasz Index")
```



Based on the elbow plot, validation metrics, and interpretability of the cluster sizes, $K = 5$ was selected for the final K-means model. The argument `nstart = 25` was used so that K-means would try multiple random starting points and reduce the chance of selecting a poor local solution.

Final K-means Model and Evaluation

```
set.seed(123)
kmeans_model <- kmeans(data_scaled, centers = 5, nstart = 25)
data_cluster$kmeans_cluster <- as.factor(kmeans_model$cluster)

table(data_cluster$kmeans_cluster)

##
##   1    2    3    4    5
## 1317  314 1709 1598  890

clusters <- kmeans_model$cluster
sil_kmeans <- silhouette(clusters, dist(data_scaled))
kmeans_silhouette <- mean(sil_kmeans[, 3])

kmeans_db <- intCriteria(as.matrix(data_scaled), clusters,
"Davies_Bouldin")$davies_bouldin
kmeans_ch <- intCriteria(as.matrix(data_scaled), clusters,
"Calinski_Harabasz")$calinski_harabasz

kmeans_metrics <- data.frame(
  Method = "K-means",
```

```

    Silhouette = round(kmeans_silhouette, 4),
    Davies_Bouldin = round(kmeans_db, 4),
    Calinski_Harabasz = round(kmeans_ch, 4)
)

```

```

kable(kmeans_metrics, caption = "Final K-means Evaluation Metrics")

```

Final K-means Evaluation Metrics

Method	Silhouette	Davies_Bouldin	Calinski_Harabasz
--------	------------	----------------	-------------------

K-means	0.2153	1.4129	1205.1
---------	--------	--------	--------

```

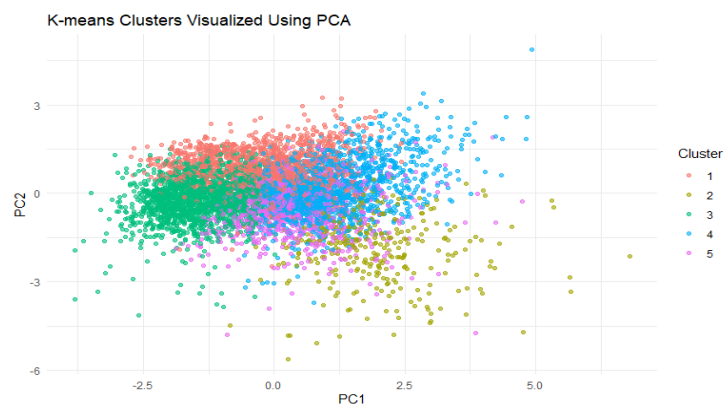
pca_data$kmeans_cluster <- data_cluster$kmeans_cluster

```

```

ggplot(pca_data, aes(x = PC1, y = PC2, color = kmeans_cluster)) +
  geom_point(alpha = 0.6) +
  theme_minimal() +
  labs(title = "K-means Clusters Visualized Using PCA", x = "PC1", y = "PC2",
color = "Cluster")

```



```

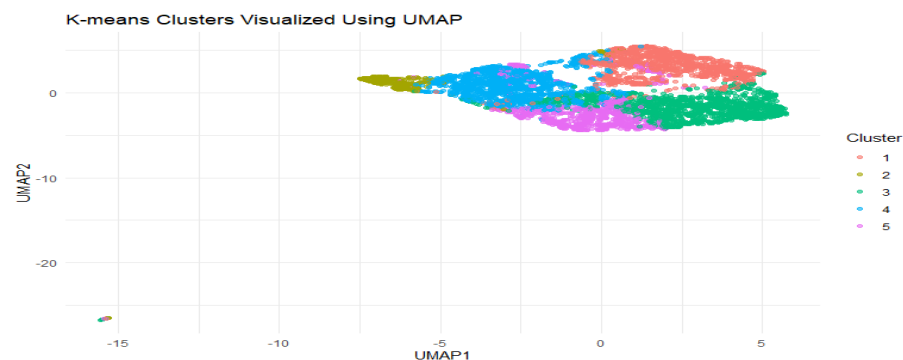
umap_data$kmeans_cluster <- data_cluster$kmeans_cluster

```

```

ggplot(umap_data, aes(x = UMAP1, y = UMAP2, color = kmeans_cluster)) +
  geom_point(alpha = 0.6) +
  theme_minimal() +
  labs(title = "K-means Clusters Visualized Using UMAP", x = "UMAP1", y =
"UMAP2", color = "Cluster")

```



Model Comparison and Final Method Selection

The three clustering methods were compared using internal validation metrics and practical interpretability. The Silhouette Score measures how well observations fit within their assigned cluster compared with other clusters, with higher values indicating better separation. The Davies-Bouldin Index measures average cluster similarity, with lower values indicating better separation. The Calinski-Harabasz Index compares between-cluster dispersion to within-cluster dispersion, with higher values indicating better-defined clusters.

```
method_comparison <- bind_rows(kmeans_metrics, dbscan_metrics, gmm_metrics)

kable(method_comparison, caption = "Comparison of Clustering Methods")
```

Comparison of Clustering Methods

Method	Silhouette	Davies_Bouldin	Calinski_Harabasz
K-means	0.2153	1.4129	1205.0999
DBSCAN	0.4811	0.7432	96.6371
GMM	0.1746	1.9078	991.7870

Because the purpose of this project was to identify meaningful and interpretable health profiles, K-means was selected as the final clustering method. It provided a strong balance between statistical performance, cluster size distribution, and practical interpretation. DBSCAN was useful for identifying noise and unusual observations, while GMM was useful for examining overlapping probability-based group structure.

K-means Cluster Profiling and Discussion

The final K-means model was profiled because it was selected as the best method for this project. The profiling analysis uses summary statistics, sex distribution, feature comparison plots, boxplots, a heatmap, and a parallel coordinate plot. Waist was not included in the cluster profiling because it was removed from the clustering model due to high correlation with BMI. Sex was included only as a descriptive profiling variable.

```
kprof <- data_imputed %>%
  dplyr::select(age, sex, income_pir, bmi, sys_bp, dia_bp, hba1c)

# Add K-means cluster labels
kprof$kmeans_cluster <- as.factor(kmeans_model$cluster)

# Label sex variable for interpretation
kprof$sex <- as.character(kprof$sex)
kprof$sex <- case_when(
  kprof$sex %in% c("1", "Male", "male", "MALE") ~ "Male",
```

```

kprof$sex %in% c("2", "Female", "female", "FEMALE") ~ "Female",
TRUE ~ kprof$sex
)
kprof$sex <- as.factor(kprof$sex)

profile_vars <- c("age", "income_pir", "bmi", "sys_bp", "dia_bp", "hba1c")

```

Cluster Size and Percentage

```

cluster_size <- kprof %>%
  group_by(kmeans_cluster) %>%
  summarise(n = n(), .groups = "drop") %>%
  mutate(percent = round(n / sum(n) * 100, 2))

kable(cluster_size, caption = "K-means Cluster Size and Percentage")

```

K-means Cluster Size and Percentage

kmeans_cluster	n	percent
1	1317	22.60
2	314	5.39
3	1709	29.32
4	1598	27.42
5	890	15.27

Summary Statistics and Cluster Profiles

```

cluster_profile <- kprof %>%
  group_by(kmeans_cluster) %>%
  summarise(
    n = n(),
    mean_age = mean(age, na.rm = TRUE),
    mean_income_pir = mean(income_pir, na.rm = TRUE),
    mean_bmi = mean(bmi, na.rm = TRUE),
    mean_systolic_bp = mean(sys_bp, na.rm = TRUE),
    mean_diastolic_bp = mean(dia_bp, na.rm = TRUE),
    mean_hba1c = mean(hba1c, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(across(where(is.numeric), ~round(.x, 2)))

kable(cluster_profile, caption = "K-means Cluster Profile Summary")

```

K-means Cluster Profile Summary

kmeans_cl uster	mean n	mean_inco _age	mean_inco me_pir	mean _bmi	mean_systoli c_bp	mean_diastoli c_bp	mean_h ba1c
1	13	49.38	4.62	27.81	120.37	72.13	5.59

kmeans_cluster	n	mean_age	mean_income_pir	mean_bmi	mean_systolic_bp	mean_diastolic_bp	mean_hba1c
1	174	60.29	2.17	32.51	129.50	69.73	9.23
2	170	30.48	1.66	25.67	111.58	65.46	5.31
3	159	66.01	1.89	27.80	141.16	75.24	5.85
4	890	43.77	1.90	40.97	125.59	74.69	5.74

Sex Distribution by Cluster

```
sex_profile <- kprof %>%
  group_by(kmeans_cluster, sex) %>%
  summarise(n = n(), .groups = "drop") %>%
  group_by(kmeans_cluster) %>%
  mutate(percent = round(n / sum(n) * 100, 2)) %>%
  ungroup()
```

```
kable(sex_profile, caption = "Sex Distribution by K-means Cluster")
```

Sex Distribution by K-means Cluster

kmeans_cluster	sex	n	percent
1	Female	645	48.97
1	Male	672	51.03
2	Female	141	44.90
2	Male	173	55.10
3	Female	922	53.95
3	Male	787	46.05
4	Female	779	48.75
4	Male	819	51.25
5	Female	522	58.65
5	Male	368	41.35

```
ggplot(sex_profile,
  aes(x = kmeans_cluster, y = percent, fill = sex)) +
  geom_col(position = "dodge") +
  theme_minimal() +
  labs(
    title = "Sex Distribution Across K-means Clusters",
    x = "Cluster",
```

```
y = "Percent",
fill = "Sex"
)
```

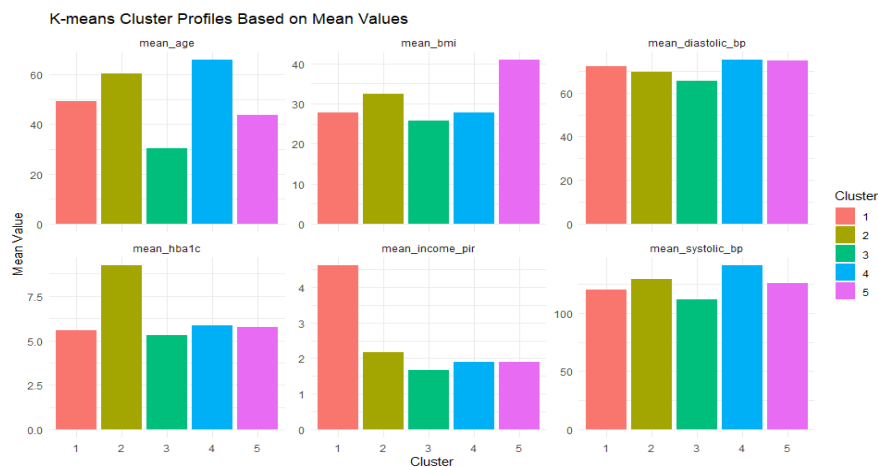


Sex was not used to create the clusters, but it was used to describe the clusters after the K-means model was created. This helps determine whether some health profiles are more common among male or female participants.

Feature Comparison Bar Plot

```
cluster_profile_long <- cluster_profile %>%
  dplyr::select(-n) %>%
  pivot_longer(cols = -kmeans_cluster, names_to = "feature", values_to =
"mean_value")

ggplot(cluster_profile_long, aes(x = kmeans_cluster, y = mean_value, fill =
kmeans_cluster)) +
  geom_col() +
  facet_wrap(~ feature, scales = "free_y") +
  theme_minimal() +
  labs(title = "K-means Cluster Profiles Based on Mean Values", x =
"Cluster", y = "Mean Value", fill = "Cluster")
```



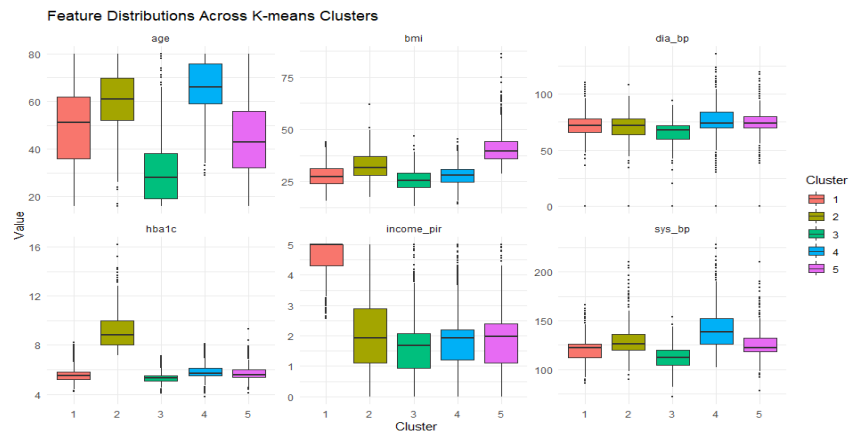
Feature Distribution Boxplots

```
kprof %>%
  dplyr::select(all_of(profile_vars), kmeans_cluster) %>%
```

```

pivot_longer(cols = all_of(profile_vars), names_to = "feature", values_to =
"value") %>%
  ggplot(aes(x = kmeans_cluster, y = value, fill = kmeans_cluster)) +
  geom_boxplot(outlier.size = 0.5) +
  facet_wrap(~ feature, scales = "free_y") +
  theme_minimal() +
  labs(title = "Feature Distributions Across K-means Clusters", x =
"Cluster", y = "Value", fill = "Cluster")

```



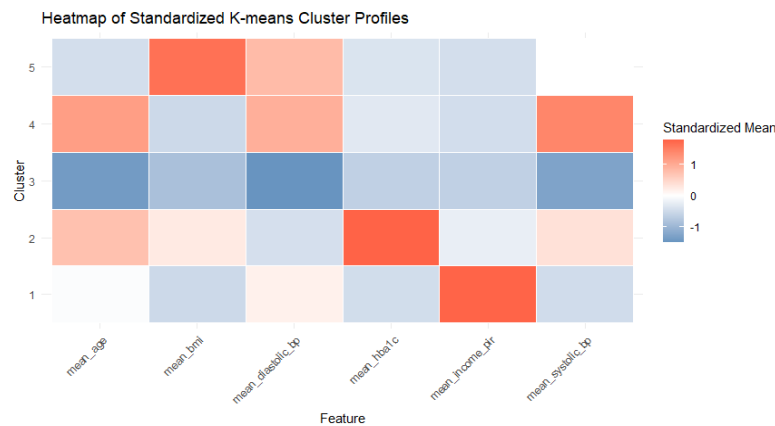
Heatmap of Standardized Cluster Profiles

```

heatmap_data <- cluster_profile %>%
  dplyr::select(-n) %>%
  pivot_longer(cols = -kmeans_cluster, names_to = "feature", values_to =
"mean_value") %>%
  group_by(feature) %>%
  mutate(standardized_mean = as.numeric(scale(mean_value))) %>%
  ungroup()

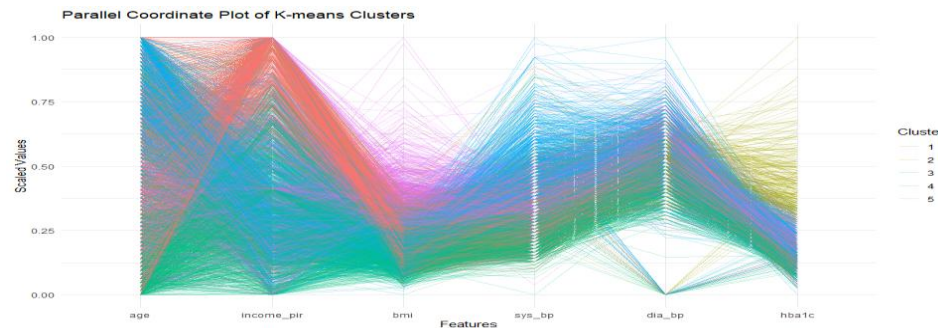
ggplot(heatmap_data, aes(x = feature, y = kmeans_cluster, fill =
standardized_mean)) +
  geom_tile(color = "white") +
  scale_fill_gradient2(low = "steelblue", mid = "white", high = "tomato",
midpoint = 0) +
  theme_minimal() +
  labs(title = "Heatmap of Standardized K-means Cluster Profiles", x =
"Feature", y = "Cluster", fill = "Standardized Mean") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

```



Parallel Coordinate Plot

```
kprof %>%
  dplyr::select(age, income_pir, bmi, sys_bp, dia_bp, hba1c, kmeans_cluster)
%>%
  ggparcoord(columns = 1:6, groupColumn = 7, scale = "uniminmax", alphas =
    0.25) +
  theme_minimal() +
  labs(title = "Parallel Coordinate Plot of K-means Clusters", x =
    "Features", y = "Scaled Values", colour = "Cluster")
```



Summary of Visualizations Created

This analysis created several visualizations to support clustering interpretation. Histograms and boxplots were used during exploratory data analysis to examine variable distributions and possible outliers. A missing-value bar chart was used to show the amount of missing data by variable. A correlation matrix was used to guide feature selection. PCA and UMAP plots were used to visualize the overall structure of the data and the clusters produced by K-means, DBSCAN, and GMM. Validation metric plots were used to support parameter choices. Silhouette plots were used to assess cluster separation. Finally, K-means cluster profiles were visualized using bar plots, boxplots, a heatmap, sex distribution plots, and a parallel coordinate plot.

Conclusion

Overall, the analysis successfully applied and compared three clustering methods to NHANES health data. The preprocessing steps included merging NHANES files, selecting relevant health variables, checking missing values, applying median imputation, removing redundant variables, and scaling the final clustering variables. K-means, DBSCAN, and GMM were then compared using internal validation metrics and practical interpretability. K-means was selected as the final method because it produced interpretable health-related clusters that were useful for profiling participants based on age, income, BMI, blood pressure, and HbA1c. Sex was not used in creating the clusters, but it was included in the final profiling stage to describe the demographic composition of each cluster.